

wav2vec 2.0 : architecture et application à Sound-Embeddings

Extrait de carnet de recherche

Dimitry Popov

Août 2026

1 Wav2vec 2.0

Wav2vec 2.0 est un modèle de reconnaissance vocale auto-supervisé composé de 3 blocs :

- un **encodeur** f qui transforme un extrait audio brut en représentation latente continue ;
- un **réseau de contexte** g qui transforme la représentation latente en représentation conceptualisée ;
- un **module de quantification** h qui discrétise les z_i pour la tâche contrastive.

A) Encodeur

L'encodeur $f : X \rightarrow Z$, $Z = (z_1, \dots, z_T)$ avec $z_i \in \mathbb{R}^k$, est composé de 7 blocs, chacun suivant la même architecture, avec un stride $s \in \mathbb{N}^*$, une taille de kernel $d \in \mathbb{N}^*$, et un nombre de canaux $f = 512$ choisis empiriquement (dans le papier : $s = 5$, $d = 10$ pour le premier bloc, puis des valeurs décroissantes pour les blocs suivants).

Bloc 1. L'idée est de faire des convolutions 1D avec un kernel donné, décalé à chaque fois d'un stride donné, pour permettre au modèle d'apprendre des motifs locaux. Pour un canal, cela se traduit par :

$$\forall i \in \{0, \dots, T-1\}, \quad y_i = \sum_{k=0}^{d-1} x_{s \cdot i + k} w_k.$$

Pour le canal 0, on obtient un vecteur $(y_0^{(0)}, y_1^{(0)}, \dots, y_{T-1}^{(0)})$ de dimension $1 \times T$. En ajoutant les 511 autres canaux, on obtient une matrice

$$\tilde{z} = \begin{pmatrix} y_0^{(0)} & y_1^{(0)} & \cdots & y_{T-1}^{(0)} \\ y_0^{(1)} & y_1^{(1)} & \cdots & y_{T-1}^{(1)} \\ \vdots & & \ddots & \vdots \\ y_0^{(511)} & y_1^{(511)} & \cdots & y_{T-1}^{(511)} \end{pmatrix}, \quad \dim(\tilde{z}) = 512 \times T.$$

On normalise ensuite chaque canal (mean = 0, variance = 1) :

$$\text{out}_i^{(f)} = \gamma^{(f)} \odot \frac{\tilde{z}_i^{(f)} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta^{(f)},$$

où γ et β sont des vecteurs appris, permettant une mise à l'échelle après normalisation. On applique enfin $\text{GELU}(x)$ sur $\text{out}_i^{(f)}$, et on remplace $\tilde{z}_i^{(f)}$ par le résultat dans $\tilde{z}$.

Remarque (définition) : GELU (Gaussian Error Linear Unit) est la non-linéarité utilisée à travers tout le modèle (encodeur, réseau de contexte, FFN, décodeur de Whisper). Elle est définie par

$$\text{GELU}(x) = x \Phi(x),$$

où Φ est la fonction de répartition de la loi normale centrée réduite, $\Phi(x) = \frac{1}{2} \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right)$. Intuitivement, GELU pondère x par la probabilité qu'une variable gaussienne standard soit inférieure à x :

contrairement à ReLU qui coupe brutalement à 0, GELU atténue progressivement les valeurs négatives plutôt que de les annuler complètement, ce qui la rend différentiable partout. En pratique, on utilise l'approximation

$$\text{GELU}(x) \approx 0.5 x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right) \right),$$

moins coûteuse à calculer que la fonction d'erreur exacte.

La matrice en fin de Bloc 1 est de dimension $512 \times T$. On la donne en entrée du Bloc 2, avec un stride et un kernel différents, et ainsi de suite jusqu'au Bloc 7.

En sortie du Bloc 7 : 49 vecteurs de 512 composantes chacun, on a donc transformé notre signal de 16 kHz en un signal de 49 Hz sans perte d'information utile.

À partir de $Z = (z_1, \dots, z_T)$, on va maintenant :

- créer un réseau de contexte ;
- quantifier les données.

B) Réseau de contexte

Étape 1 : transformation linéaire. Pour des considérations empiriques, on choisit de prendre en entrée du transformateur une matrice à 768 lignes. D'où la transformation :

$$z'_i = W z_i + b, \quad W \in \mathbb{R}^{768 \times 512}, \quad b \in \mathbb{R}^{768},$$

ainsi $\dim(Z') = 768 \times T$.

Étape 2 : masquage. On prend aléatoirement des colonnes Δ_i et on les masque en les remplaçant par des colonnes apprises m_i .

Étape 3 : Transformer. On pose U la matrice avec les colonnes masquées, $\dim(U) = 768 \times T$.

Étape 3.1 : embedding positionnel convolutif. Problèmes à résoudre :

- l'attention est équivariante par permutation ;
- chaque colonne ignore, seule, sa position dans la séquence.

L'idée est de tamponner chaque colonne avec une « signature » de son voisinage temporel local. On fait donc une convolution de stride 1 et de kernel 128. Pour économiser du calcul, on découpe les canaux en 16 paquets de 48 canaux (choix empirique). On pose P ce que la convolution renvoie, puis :

$$U' = U + \text{GELU}(P).$$

Étape 3.2 : Bloc 1 du transformateur. On pose $H_0 = U'$, $\dim(H_0) = 768 \times T$. On découpe la matrice en 8 têtes de 96 composantes chacune. Pour chaque vecteur colonne x_i de la couche courante, on calcule 3 projections linéaires apprises :

$$R_i = W_R x_i, \quad K_i = W_K x_i, \quad V_i = W_V x_i,$$

respectivement Query, Key, Value. On construit le score entre ce que i cherche et ce que j offre par produit scalaire :

$$s_{ij} = \frac{\langle R_i, K_j \rangle}{\sqrt{96}}, \quad \alpha_{ij} = \text{softmax}_j(s_{ij}), \quad \text{out}_i = \sum_j \alpha_{ij} V_j.$$

Pour les 8 têtes :

$$\nabla_i = [\text{out}_i^1, \dots, \text{out}_i^8], \quad \dim(\nabla) = 768 \times T.$$

On calcule :

$$a_i = W_O \nabla_i, \quad W_O \in \mathbb{R}^{768 \times 768}, \quad b_i = \text{LayerNorm}(a_i + x_i),$$

où x_i désigne l'entrée de la couche courante (résidu), pour le premier bloc, x_i correspond à la colonne i de H_0 .

On fait ensuite un Feed-Forward-Network (FFN), $768 \rightarrow 3072 \xrightarrow{\text{GELU}} 3072 \rightarrow 768$:

$$f_i = \text{FFN}(b_i) = W_2 \cdot \text{GELU}(W_1 b_i + c_1) + c_2, \quad H_1(i) = \text{LayerNorm}(f_i + b_i).$$

On répète ce bloc jusqu'à obtenir H_{12} . On pose $C = H_{12}$, la représentation conceptualisée produite par le transformer.

Tâche contrastive

Soit $t \in \{1, \dots, T\} \cap \{\text{positions masquées}\}$:

- c_t : représentation conceptualisée produite par le transformer (colonne t de C) ;
- q_t : représentation produite par le quantizer.

L'idée est que le modèle doit s'entraîner à choisir le bon q_t parmi $K + 1$ candidats, à l'aide de la similarité cosinus :

$$\text{sim}(c_t, \tilde{q}) = \frac{\langle c_t, \tilde{q} \rangle}{\|c_t\| \|\tilde{q}\|}.$$

On utilise la Contrastive Loss :

$$m = -\log \left(\frac{\exp(\text{sim}(c_t, q_t)/\kappa)}{\sum_{\tilde{q} \in Q_t} \exp(\text{sim}(c_t, \tilde{q})/\kappa)} \right),$$

où Q_t contient q_t et K distracteurs tirés des autres positions masquées de l'énoncé.

C) Quantizer

Rappel : le rôle du quantizer h est de discrétiser z_i (sortie de l'encodeur, avant masquage) pour produire q_i , comparé à c_t dans la tâche contrastive.

Étape 1 : projection vers les logits. On pose

$$l_i = W_l z_i + b_l, \quad l_i \in \mathbb{R}^{G \times V},$$

avec G le nombre de codebooks (groupes) et V le nombre d'entrées par codebook. Choix empirique du papier : $G = 2$, $V = 320$.

Étape 2 : Gumbel-softmax. Problème : un argmax n'est pas différentiable, on ne peut donc pas sélectionner directement l'entrée du codebook et rétropropager le gradient à travers cette sélection.

L'idée est d'utiliser le Gumbel-softmax trick : on transforme la sélection discrète en un échantillonnage différentiable. On tire un bruit de Gumbel pour chaque entrée :

$$n_{g,v} = -\log(-\log(u_{g,v})), \quad u_{g,v} \sim \mathcal{U}(0, 1).$$

On calcule :

$$p_{g,v} = \frac{\exp((l_{g,v} + n_{g,v})/\tau)}{\sum_{k=1}^V \exp((l_{g,k} + n_{g,k})/\tau)}.$$

En forward, on prend le vecteur one-hot dur : $e_g = \text{onehot}(\arg \max_v p_{g,v})$. En backward, on remplace le gradient de e_g par celui de $p_{g,\cdot}$ (straight-through estimator) : le forward est discret, le backward est doux.

Étape 3 : sélection dans les codebooks. Pour chaque groupe g , on pose un codebook appris $C_g \in \mathbb{R}^{V \times (d/G)}$. La sortie du groupe g est :

$$o_g = e_g^\top C_g \in \mathbb{R}^{d/G}.$$

On concatène les G groupes :

$$\tilde{o}_i = [o_1; o_2; \dots; o_G] \in \mathbb{R}^d.$$

On applique une dernière transformation linéaire pour obtenir q_i à la dimension utilisée dans la loss contrastive ($d_q = 256$ dans le papier) :

$$q_i = W_q \tilde{o}_i, \quad W_q \in \mathbb{R}^{d_q \times d}.$$

Étape 4 : diversity loss. Problème : sans contrainte, rien n’empêche le modèle de toujours choisir les mêmes entrées de codebook (*collapse*), ce qui réduit fortement la capacité de représentation du quantizer.

Sur un batch de taille B , on pose la distribution moyenne d’utilisation d’un groupe g (moyenne des probabilités softmax, pas des sélections dures) :

$$\bar{p}_{g,v} = \frac{1}{B} \sum_{i=1}^B p_{g,v}^{(i)}.$$

On définit la diversity loss :

$$L_d = \frac{1}{GV} \sum_{g=1}^G \sum_{v=1}^V \bar{p}_{g,v} \log(\bar{p}_{g,v}).$$

Minimiser L_d revient à maximiser l’entropie de $\bar{p}_g$, donc à pousser le modèle vers une utilisation uniforme des V entrées de chaque codebook.

Étape 5 : température. τ est annealé linéairement de 2 à 0.5 pendant l’entraînement : échantillonnage doux (exploration) en début d’entraînement, quasi-discret (exploitation) en fin d’entraînement.

Bilan : q_i (Étape 3) est utilisé comme $\tilde{q}$ dans $\text{sim}(c_t, \tilde{q})$ et dans la loss contrastive m définie plus haut, en compétition avec K distracteurs tirés des autres positions masquées de l’énoncé.

Synthèse : loss totale et rôle des 3 blocs

On a maintenant décrit les 3 blocs de Wav2vec 2.0 : l’encodeur f (Partie A), le réseau de contexte g (Partie B), et le quantizer h (Partie C). Chacun a un rôle précis dans l’objectif d’entraînement.

Problème : ces 3 blocs ne sont pas entraînés séparément, ils doivent l’être conjointement, avec un seul objectif à optimiser.

L’idée est de combiner la contrastive loss m (qui force c_t à se rapprocher de q_t et à s’éloigner des distracteurs) avec la diversity loss L_d (qui empêche le quantizer de s’effondrer sur un petit sous-ensemble de codes) :

$$\mathcal{L} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} m_t + \alpha L_d,$$

où $\mathcal{T}$ est l’ensemble des positions masquées de l’énoncé, et α un hyperparamètre pondérant la diversity loss face à la contrastive loss.

En reliant les 3 blocs :

- f transforme X (signal brut à 16 kHz) en $Z = (z_1, \dots, z_T)$ (représentation latente continue, 49 Hz) ;
- une copie de Z est masquée puis envoyée dans g , qui produit $C = H_{12}$, les représentations conceptualisées c_t pour chaque position masquée t ;
- la même Z (non masquée) est envoyée dans h , qui produit les cibles discrètes q_t ;
- la loss $\mathcal{L}$ compare c_t (ce que le contexte, à partir des voisins non masqués, *devine*) à q_t (ce que l’encodeur a *réellement* produit à cette position, discrétisé).

Autrement dit : le réseau de contexte apprend à reconstruire, dans un espace discret appris par le quantizer, l’information qui a été masquée en entrée. C’est cette tâche auto-supervisée qui permet à Wav2vec 2.0 de pré-entraîner f et g sans aucune transcription, sur de la parole non-annotée, exactement le type de données (X heures d’*untranscribed speech*) mentionné dans la Tâche 1 pour le calcul de l’ATDS.

2 Du quantizer au vocabulaire : Sound-Embeddings

Les trois blocs décrits ci-dessus sont entraînés conjointement par la loss $\mathcal{L} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} m_t + \alpha L_d$. Une fois cette minimisation terminée, la tâche contrastive est abandonnée : on jette le quantizer h , on

gèle f et g , et on ne conserve que les représentations produites en cours de route. Ces douze couches $H_1, \dots, H_{12}$ n'étaient l'objectif d'aucune loss elles sont un sous-produit de l'entraînement.

Le projet Sound-Embeddings part de cette observation et pose une question que le papier ne traite pas : ces représentations, apprises exclusivement sur de la parole non annotée, encodent-elles quelque chose d'exploitable sur de la *musique* ? Et si oui, à quelle profondeur ?

A) Deux quantifications, deux endroits

La démarche du projet consiste à discrétiser une représentation continue par k -means, ce qui rappelle immédiatement le quantizer de la Partie C. La parenté est réelle mais les deux opérations ne portent pas sur le même objet, et c'est précisément ce décalage qui fait l'intérêt de l'expérience.

	Quantizer h	k -means du projet
Entrée	z_i , sortie de f	H_ℓ , sortie du bloc ℓ de g
Contexte	aucun	attention sur toute la fenêtre
Apprentissage	conjoint (Gumbel-softmax)	postérieur, modèle gelé
Codes	$G = 2$ groupes de $V = 320$	un codebook de K entrées
Objectif	cible de la loss contrastive	représenter un morceau entier

Le quantizer opère sur Z , avant tout mécanisme d'attention : ses codes q_i décrivent une trame isolée. Le k -means du projet opère sur H_ℓ , donc après que le transformer a mélangé l'information de toute la fenêtre. Les pseudo-tokens obtenus ne sont donc pas une réimplémentation des q_i : ils discrétisent une représentation que le quantizer n'a jamais vue.

Formellement, on apprend un codebook $C \in \mathbb{R}^{K \times 768}$ par k -means sur un échantillon de trames issues du corpus, puis chaque trame reçoit le code du centroïde le plus proche :

$$\text{token}(h) = \arg \min_{j \in \{1, \dots, K\}} \|h - C_j\|^2 = \arg \min_j (\|C_j\|^2 - 2 \langle h, C_j \rangle),$$

le terme $\|h\|^2$ étant constant en j . Un morceau de n trames devient une suite de n symboles, puis un vecteur de proportions $p \in \mathbb{R}^K$ avec $p_j = \frac{1}{n} \#\{i : \text{token}(h_i) = j\}$. Deux morceaux sont comparés par $\cos(p^{(a)}, p^{(b)})$.

B) Pourquoi la couche 2

Le choix de ℓ n'est pas neutre. La loss contrastive porte directement sur H_{12} , qui est donc la couche la plus spécialisée à la structure de la parole ; les couches basses, elles, restent proches du signal acoustique.

L'expérience a retenu $\ell = 2$, ce qui suggère que le transfert vers la musique passe par l'acoustique et non par le linguistique. Deux observations le confirment. En recollant l'audio des trames assignées à un même token, on entend des timbres identifiables, un synthétiseur, une famille percussive et non des unités sub-phonémiques.

C) Une uniformité non contrainte

La Partie C introduit la diversity loss L_d pour empêcher le quantizer de s'effondrer sur un petit sous-ensemble de codes : minimiser L_d revient à maximiser l'entropie de $\bar{p}_g$, donc à forcer une utilisation uniforme des V entrées de chaque codebook.

Le k -means du projet n'impose aucune contrainte de ce type. Il minimise uniquement l'inertie intra-cluster. Pourtant, la distribution d'usage des pseudo-tokens observée sur un album est remarquablement plate : le token le plus fréquent représente 3,7 % des trames, le plus rare 0,6 %, là où une distribution uniforme donnerait 2 % pour $K = 50$.

Deux lectures restent possibles, et les mesures actuelles ne permettent pas de les départager. Ou bien l'espace H_2 est peu modal, et le k -means découpe un continuum sans frontières nettes, auquel cas l'uniformité est un artefact de la méthode, le k -means équilibrant naturellement les régions denses. Ou bien cette uniformité est héritée : L_d a façonné en amont un espace de représentation dont l'occupation est déjà régulière. Trancher demanderait de comparer la distribution obtenue sur H_2 à celle obtenue sur une couche haute, et de mesurer la persistance temporelle des tokens.

D) Ce que l'expérience a montré

Deux hypothèses ont été testées sur un corpus de 8000 morceaux répartis en huit genres, en mesurant la précision@5 : pour chaque morceau interrogé, la proportion de ses cinq plus proches voisins qui partagent son genre du coup une requête dont trois voisins sur cinq sont du bon genre compte pour $3/5$, moyennée sur l'ensemble des requêtes. Un moteur renvoyant des morceaux au hasard obtiendrait $1/8 = 0,125$, les huit genres étant équilibrés.

La première hypothèse, issue de la recherche d'information textuelle, consistait à pondérer chaque token par son IDF (Spärck Jones, 1972), afin d'atténuer le poids des unités présentes partout. À $K = 50$, l'effet est nul. Un token acoustique fréquent décrit malgré tout du son, et le pénaliser retire autant de signal que de bruit.

La seconde portait sur la granularité du vocabulaire. Le passage de $K = 50$ à $K = 256$ fait passer la précision de 0,31 à 0,36, et la pondération IDF ne devient légèrement utile qu'à cette granularité lorsque les unités sont assez spécifiques pour qu'une pondération ait quelque chose à distinguer.

Autrement dit : la taille du codebook comptait davantage que la façon de pondérer ses unités. Ce résultat n'était pas prédit par l'analogie textuelle qui avait motivé la première hypothèse, et il rejoint le constat de la section précédente avec $K = 50$, la discrétisation était trop grossière pour que la structure de H_2 soit exploitable.